Singly Linked Lists in C++

Basic, Advanced Operations and File Handling

Le Nhut Nam

Department of Computer Science

December 17, 2025

Why Linked Lists?

- Arrays have fixed size
- Insertions and deletions are costly
- Linked lists use dynamic memory allocation

Key Idea

A linked list is a collection of nodes connected using pointers.

Singly Linked List

- Each node contains:
  - Data
  - Pointer to the next node
- Last node points to `NULL`

$$\boxed{10} \rightarrow \boxed{20} \rightarrow \boxed{30} \rightarrow \texttt{NULL}$$

Node Structure

Space complexity: $O(1)$

```
struct Node {
    int data;
    Node* next;
};
```

Basic Operations

- Insert at beginning
- Insert at end
- Delete a node
- Traverse the list

Insert at Beginning

```
void insertFront(Node*& head, int value) {
    Node* newNode = new Node;
    newNode->data = value;
    newNode->next = head;
    head = newNode;
}
```

Insert at End

Time complexity: $O(n)$

```
void insertEnd(Node*& head, int value) {
    Node* newNode = new Node;
    newNode->data = value;
    newNode->next = NULL;

    if (head == NULL) {
        head = newNode;
        return;
    }

    Node* temp = head;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = newNode;
}
```

Traversal

```
void display(Node* head) {
    Node* temp = head;
    while (temp != NULL) {
        cout << temp->data << " -> ";
        temp = temp->next;
    }
    cout << "NULL" << endl;
}
```

Delete a Node

```
void deleteValue(Node*& head, int value) {
    if (head == NULL) return;

    if (head->data == value) {
        Node* temp = head;
        head = head->next;
        delete temp;
        return;
    }
```

Delete a Node (Cont)

```
    Node* curr = head;
    while (curr->next != NULL &&
           curr->next->data != value) {
        curr = curr->next;
    }

    if (curr->next != NULL) {
        Node* temp = curr->next;
        curr->next = temp->next;
        delete temp;
    }
}
```

Advanced Operations

- Reverse a linked list
- Find middle element
- Detect cycles
- Merge sorted lists

Reverse a Linked List

```
Node* reverse(Node* head) {
    Node* prev = NULL;
    Node* curr = head;

    while (curr != NULL) {
        Node* next = curr->next;
        curr->next = prev;
        prev = curr;
        curr = next;
    }
    return prev;
}
```

Time: $O(n)$, Space: $O(1)$

Linked List + File Handling

Goal

Read integers from a file and store them in a linked list.

Load Linked List from File

```
#include <fstream>

Node* loadFromFile(const string& filename) {
    ifstream inFile(filename);
    Node* head = NULL;
    int x;

    while (inFile >> x) {
        insertEnd(head, x);
    }

    inFile.close();
    return head;
}
```

Save Linked List to File

```cpp
#include <fstream>

void saveToFile(Node* head, const string& filename) {
    ofstream outFile(filename);
    Node* temp = head;

    while (temp != NULL) {
        outFile << temp->data << " ";
        temp = temp->next;
    }

    outFile.close();
}
```

Common Mistakes

- Forgetting `delete` (memory leak)
- Losing the head pointer
- Infinite loops
- Not checking file open status

Summary

- Linked lists are dynamic data structures
- Efficient insertion and deletion
- Advanced operations improve algorithmic thinking
- File handling enables persistent storage

References

- C++ Reference: `https://cplusplus.com/reference/`
- cppreference: `https://en.cppreference.com/w/cpp/io`
- Stroustrup, B. (2013). *The C++ Programming Language*

Thank You!

Questions?